SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1*: Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 60%

Dr Ramamoorthy

QUESTIONS: 12 Total Marks:60

Annexure A

CONCEPT MAPPING

Code of Conduct:

I ***Ranjanaa V P 192320006*** certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1. Construct a Concept Map — Software Development Life Cycle *5 Marks*

GIVEN

```
Requirement Analysis → _______ → Coding → Testing → _______ → Maintenance
```

ANSWER

Missing concept(s): Design, Deployment.

CONCEPT MAP DIAGRAM

EXPLANATION:

Design is the bridge between what the client wants and what gets built. It takes every requirement gathered during analysis and converts it into a concrete blueprint — architecture diagrams, database schema, module boundaries, and interface definitions — so that programmers know exactly what to code. **Deployment** is the stage where the fully tested software is packaged, installed, and released into the live/production environment so real users can start using it, right before ongoing maintenance begins.

The Software Development Life Cycle (SDLC) is sequential and cumulative: each phase consumes the output of the phase before it. Skipping or rushing Design usually shows up later as messy, hard-to-extend code, because Coding had no clear blueprint to follow. Skipping or rushing Deployment (e.g., no rollback plan, no environment checks) causes production failures even when Coding and Testing were done well.

WORKED EXAMPLE:

Consider a college building an online fee-payment portal. During Requirement Analysis, the team learns students need to pay fees, view receipts, and get reminders before due dates. In Design, this is converted into a database schema (Student, Payment, tables), a payment-gateway integration plan, and screen layouts. Coding implements the login page, payment form, and receipt generator. Testing checks that a payment of Rs. 5,000 correctly updates the balance and generates a valid receipt. Deployment installs the portal on the college's live server so students can actually use it, and Maintenance later adds a new payment method such as UPI when the college requests it.

ANALYSIS:

Ensures a systematic, traceable path from idea to working product. Reduces project risk by catching structural problems at the Design stage, which is far cheaper to fix than after Coding. Improves long-term software quality and maintainability because later phases inherit a solid, well-designed foundation.

Q2. Construct a Concept Map — Object-Oriented Principles

5 Marks

GIVEN

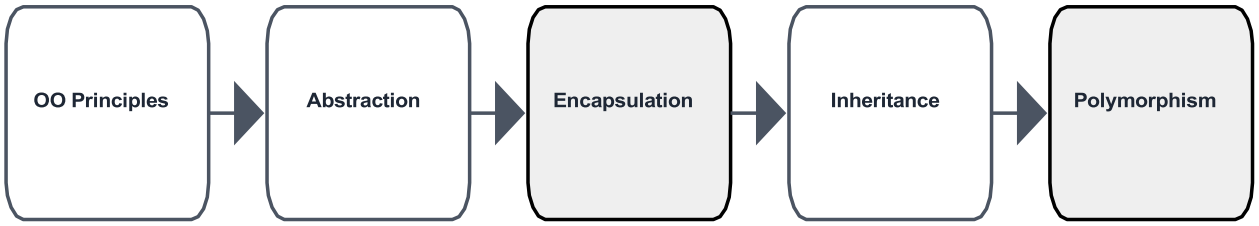
```
Object-Oriented Principles → Abstraction → _______ → Inheritance → ______
```

ANSWER

Missing concept(s): Encapsulation, Polymorphism.

CONCEPT MAP DIAGRAM

Q2. Object-Oriented Principles -
Concept Map



EXPLANATION

Encapsulation bundles an object's data (attributes) and the methods that operate on that data into a single unit, and restricts direct access to internal details using access modifiers (private/protected), exposing only what is necessary through public methods. **Polymorphism** allows a single interface (e.g., one method name) to behave differently depending on the object that invokes it — through method overriding (runtime) or method overloading (compile-time).

Abstraction hides implementation complexity and shows only essential features to the user — it answers *what* an object does, not *how*. Encapsulation protects that hidden implementation from unauthorized or accidental external modification, enforcing controlled access via getters/setters. Inheritance lets a new class (child/subclass) reuse the attributes and methods of an existing class (parent/superclass), establishing an 'is-a' relationship. Polymorphism then lets that inherited interface be used interchangeably even though the actual behavior differs per subclass — e.g., a Shape reference calling draw() behaves differently for Circle and Rectangle objects.

WORKED EXAMPLE

In a banking application, Abstraction is used when a customer calls account.withdraw(2000) without needing to know how balance checks, fraud checks, or ledger updates happen internally. Encapsulation keeps the actual balance field private, only allowing changes through the withdraw() and deposit() methods so no external code can directly set balance = 999999. Inheritance lets a SavingsAccount and a CurrentAccount class both extend a common Account class, reusing fields like accountNumber and balance. Polymorphism then allows a single calculateInterest() call on an Account reference to compute interest differently depending on whether the actual object is a SavingsAccount or a CurrentAccount.

ANALYSIS

Improves data security by preventing unauthorized access to critical fields. Enhances code reusability — inherited functionality does not need to be rewritten for every subclass. Supports flexible, extensible software design, since new object types can be added with minimal change to existing code (Open/Closed Principle).

Q3. Construct a Concept Map — Class, Object, Method, Attribute

5 Marks

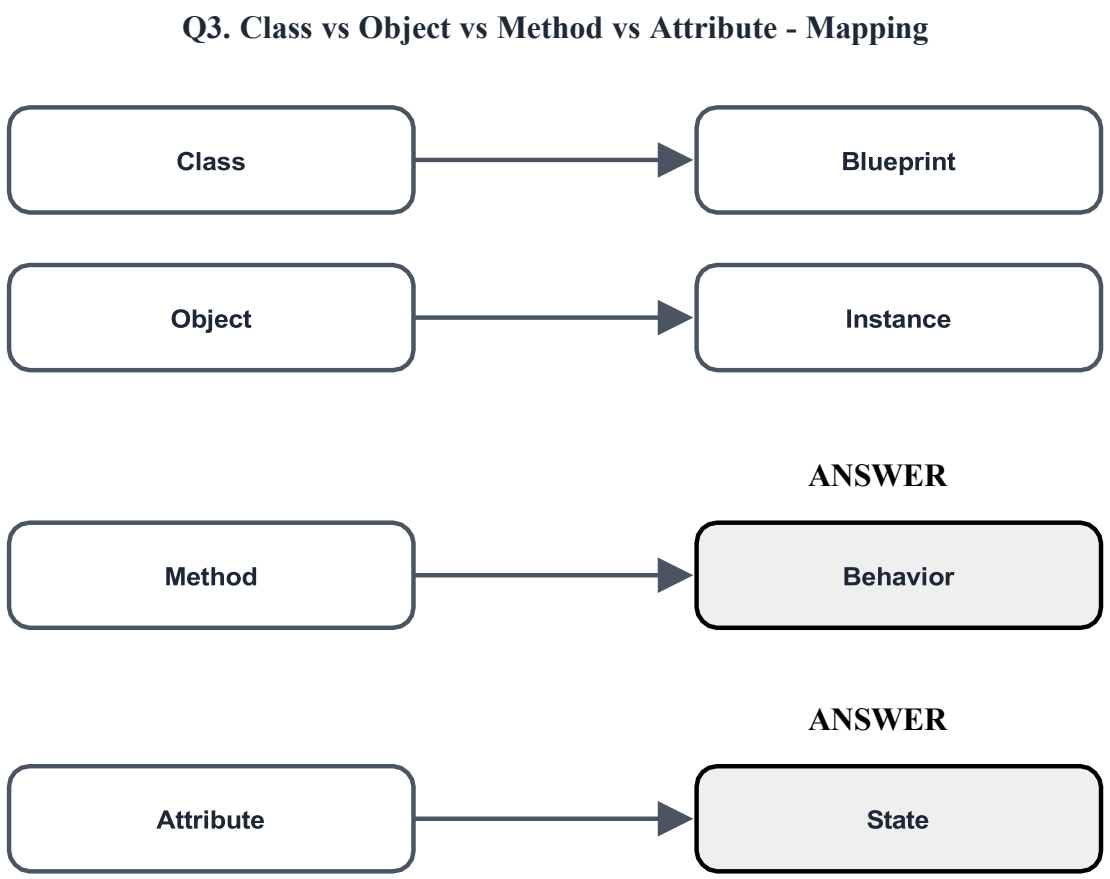
GIVEN

```
Class → Blueprint
Object → Instance
Method → _______
Attribute → _______
```

ANSWER

Missing concept(s): Behavior, State.

CONCEPT MAP DIAGRAM



EXPLANATION

A **Method** represents the **Behavior** of an object — it is a function defined inside a class that specifies an action the object can perform (e.g., a Car object's accelerate() or brake() method). An **Attribute** represents the **State** of an object — it is a variable defined inside a class that stores the current data/condition of that object at any point in time (e.g., a Car object's speed or fuelLevel).

Just as a Class is a Blueprint (a template with no memory of its own) and an Object is an Instance created from that blueprint (an actual entity occupying memory), Methods and Attributes describe the two things every real-world instance has: things it can do (behavior) and things it knows about itself (state). Together, State + Behavior fully describe an object at runtime — this pairing is the foundation of object modelling in OOAD.

WORKED EXAMPLE

For a class Student, the Class itself is only a Blueprint — it defines that every student has a name, rollNumber, and marks, and can perform actions like calculateGrade(). When the program runs student1 = new Student("Ranjanaa", "192320006", 89), an Object (Instance) is created in memory. At that moment, the Attributes name = "Ranjanaa", rollNumber = "192320006", and marks = 89 represent the object's State, while calling student1.calculateGrade() represents its Behavior, since it performs an action using that state to return a grade such as 'A'.

ANALYSIS

Helps accurately model real-world entities as software objects during analysis and design. Clearly separating state from behavior makes class responsibilities easier to define (Single Responsibility Principle). Improves overall system organization, since every class can be documented as a set of attributes (state) plus a set of methods (behavior).

Q4. A Concept Map Shows — Software Quality Attributes

5 Marks

GIVEN

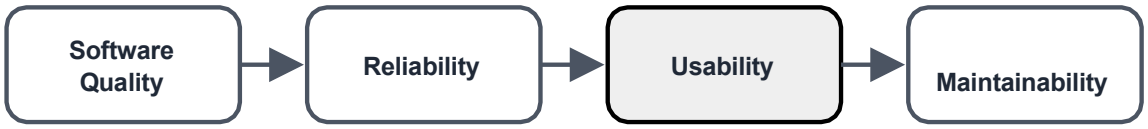
```
Software Quality → Reliability → _______ → Maintainability
```

ANSWER

Missing concept(s): Usability.

CONCEPT MAP DIAGRAM

Q4. Software Quality Attributes - Concept Map



EXPLANATION

Usability measures how easily, efficiently, and intuitively end users can learn and operate the software — covering interface clarity, learnability, and user satisfaction. It logically sits between Reliability (does the software work correctly and consistently?) and Maintainability (can the software be updated easily?) because a system can be technically correct and easy to modify, yet still fail in the market if real users find it confusing to use.

Reliable software performs its intended functions consistently, without unexpected failures, over a specified period of time. Usable software minimizes user error and training time through clear, well-designed interfaces and predictable interactions. Maintainable software is structured so that developers can locate and fix defects, or add new features, without breaking existing functionality.

WORKED EXAMPLE

Consider a food-delivery app. It is Reliable if it correctly places an order and charges the right amount every single time, without crashing during checkout. It is Usable if a first-time user can order a pizza within two minutes without needing a tutorial, because the menu, cart, and 'Place Order' button are all clearly laid out. It is Maintainable if, when the restaurant partner adds a new payment method next year, a developer can add it by editing one payment module instead of rewriting the whole checkout flow.

ANALYSIS

Improves overall user satisfaction and adoption of the product. Enhances the product's acceptance in the market compared to functionally similar but harder-to-use competitors. Directly contributes to the long-term commercial success of the software

Q5. UML Concept Map

5 Marks

GIVEN

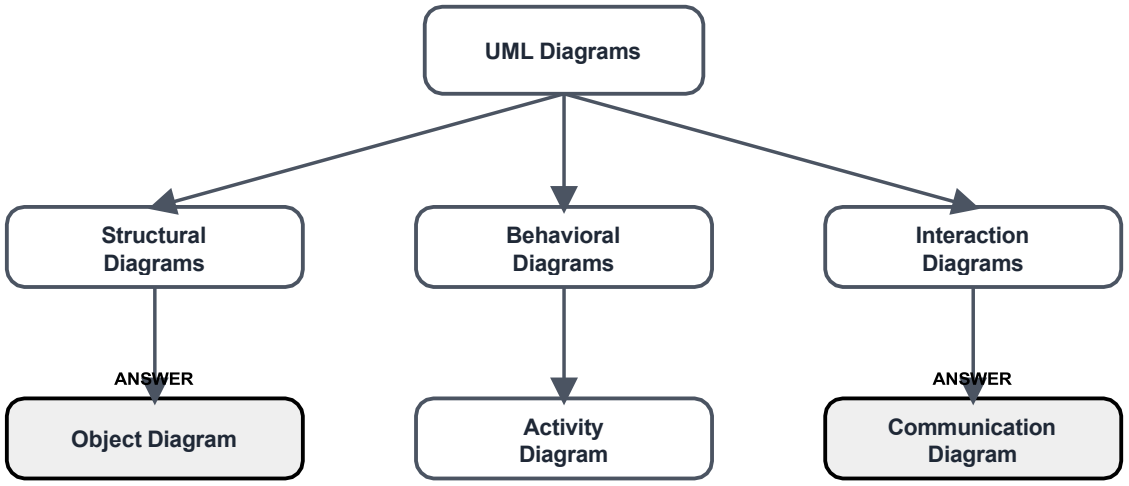
```
UML → Structural Diagrams → _______
UML → Behavioral Diagrams → Activity Diagram
UML → Interaction Diagrams → _______
```

ANSWER

Missing concept(s): Object Diagram, Communication Diagram.

CONCEPT MAP DIAGRAM

Q5. UML Diagram Concept Map



EXPLANATION

An **Object Diagram** is a Structural UML diagram that shows a snapshot of actual object instances and the links between them at a specific point in time — essentially a real-world example of a Class Diagram's structure. A **Communication Diagram** (formerly called a Collaboration Diagram) is an Interaction UML diagram that shows how objects are linked together and the sequence of messages they exchange to accomplish a task, emphasizing the relationships between objects rather than strict time ordering.

UML (Unified Modeling Language) diagrams are broadly grouped into Structural diagrams (what the system *is* — Class, Object, Component, Deployment) and Behavioral diagrams (what the system *does* — Activity, State Machine, Use Case). Interaction diagrams are actually a sub-category of Behavioral diagrams that specifically focus on how objects communicate — including Sequence Diagrams and Communication Diagrams. An Activity Diagram models the workflow/logic of a process, step by step, similar to a flowchart.

WORKED EXAMPLE

For a Library Management System: a Class Diagram (Structural) shows that a Book class has a title and isbn, and relates to a Member class. An Object Diagram (Structural) would show a concrete snapshot, e.g., Book("Data Structures", "ISBN-1234") currently linked to Member("Ranjanaa", id="192320006"). An Activity Diagram (Behavioral) models the borrowing workflow: search book -> check

availability -> issue book -> update due date. A Communication Diagram (Interaction) shows the Member object sending a borrowBook() message to the Book object, which in turn sends an updateStatus() message to the Catalog object.

ANALYSIS

Improves system visualization from multiple complementary angles — static structure and dynamic behavior. Supports object-oriented analysis by letting designers verify that the intended object collaborations actually match the class structure. Facilitates communication among stakeholders (developers, testers, clients) who may need different diagram types to understand the same system.

Q6. Concept Map with Failure Analysis — SDLC Breakdown

5 Marks

GIVEN

```
Requirements ✓ → Design ✓ → Coding ✗ → Testing ✗ → Deployment ✗
```

CONCEPT MAP DIAGRAM

Q6. SDLC Failure Point Analysis

✓ Phase completed successfully

✗ Phase failed / not reached

□ *Phase completed (check)* □ *Phase failed / blocked (cross)*

EXPLANATION

The check marks show that Requirements gathering and Design were completed successfully — the team correctly understood what to build and correctly planned how to build it. The first cross mark appears at **Coding**, meaning the failure originates there: the implementation does not correctly translate the design into working, logically correct program code. Because Coding failed, everything downstream (Testing, Deployment) is automatically blocked/marked failed too — Testing cannot verify a product that does not correctly exist yet, and a broken product cannot be safely Deployed.

WORKED EXAMPLE

Suppose a team is building a hostel room-allocation system. Requirements were gathered correctly (students need to see available rooms and book one), and the Design correctly planned

a Room table and an allocation algorithm. But during Coding, a developer implements the allocation logic with an off-by-one error, so the last room in every block is never offered to students. This is a coding defect, not a requirements or design flaw, which is exactly why the failure marker sits at the Coding phase and blocks Testing and Deployment until it is fixed.

ANALYSIS

Errors introduced at the Coding phase propagate forward and contaminate every subsequent phase. Testing cannot proceed effectively or meaningfully against buggy code, since defects mask each other and waste tester effort. Overall software quality decreases, and the project timeline slips as the team must loop back to fix Coding before moving forward again.

SOLUTION / CORRECTIVE ACTION

Review the source code line by line (peer code review) to catch logic and syntax errors early. Apply consistent coding standards and naming conventions so errors are easier to spot and code is easier to maintain. Conduct structured code inspections / static analysis before handing code over to the Testing phase, catching defects at the cheapest point to fix them.

Q7. Construct a Concept Map — Software Development Process

5 Marks

GIVEN

```
Problem Identification → _______ → Design → Implementation → ______
```

ANSWER

Missing concept(s): Requirement Analysis, Testing.

CONCEPT MAP DIAGRAM

Q7. Software Development Process - Concept Map

ANSWER **ANSWER**

□ *Given concept* □ *Answer (missing concept filled in)*

EXPLANATION

Requirement Analysis comes right after a problem is identified — it is the process of systematically gathering, studying, and documenting exactly what the users and stakeholders need the system to do, forming the foundation for Design. **Testing** comes right after Implementation — it is the process of executing the built system with the goal of finding defects and verifying that it actually satisfies the requirements gathered at the start.

Problem Identification defines the pain point or opportunity that justifies building software at all. Requirement Analysis translates that vague problem into precise, verifiable requirements (functional and non-functional). Design and Implementation then convert those requirements into an architecture and, ultimately, working code. Testing closes the loop by checking

Implementation against the original Requirement Analysis — confirming the circle from 'problem' to 'verified solution' is complete.

WORKED EXAMPLE

A hospital notices that patients wait too long to see a doctor (Problem Identification). Requirement Analysis reveals the actual need is an online token/appointment system with doctor availability shown in real time. Design lays out the database and screens for booking a token. Implementation builds the booking page and token-generation logic. Testing then verifies the built system against the original requirement — for example, confirming that booking a token for Dr. Kumar at 10 AM correctly blocks that same slot from being booked again by another patient.

ANALYSIS

Improves software quality by catching mismatches between what was built and what was actually needed. Reduces development errors and rework by validating requirements before large amounts of code are written. Ensures successful, on-target project completion that genuinely solves the identified problem.

Q8. Construct a Concept Map — Object-Oriented Features

5 Marks

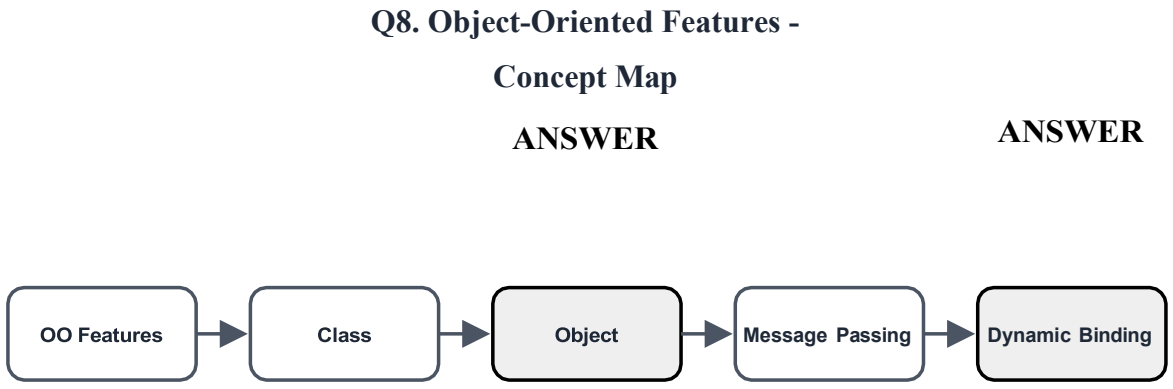
GIVEN

```
Object-Oriented Features → Class → _______ → Message Passing → _______
```

ANSWER

Missing concept(s): Object, Dynamic Binding.

CONCEPT MAP DIAGRAM



□ *Given concept* □ *Answer (missing concept filled in)*

EXPLANATION

An **Object** is a runtime instance of a Class — it occupies memory and holds actual attribute values, and it is the entity that actually sends and receives messages while a program executes. **Dynamic Binding** (also called late binding) is the mechanism by which the specific method implementation to execute is decided at runtime rather than compile time, based on the actual

object type — this is what makes runtime polymorphism possible.

A Class only becomes usable once Objects are instantiated from it; those Objects then interact with one another via Message Passing (i.e., one object calling another object's method). When that message is received, Dynamic Binding determines exactly which version of the method actually runs, based on the real (runtime) type of the receiving object, not just its declared/reference type.

WORKED EXAMPLE

In a ride-hailing app, the Class Vehicle defines general fields like plateNumber and capacity. When a specific car is registered, an Object such as vehicle1 (a Car) or vehicle2 (a Bike) is created. When a ride is requested, the app performs Message Passing by calling vehicle.calculateFare(distance) on whichever vehicle object was assigned. Dynamic Binding ensures that this single call automatically runs the Car's fare formula for vehicle1 and the Bike's fare formula for vehicle2, without the calling code needing an if-else check for vehicle type.

ANALYSIS

Supports flexibility in software design, since behavior can vary by object type without changing the calling code. Improves code maintainability, because new object types can be introduced without modifying existing message-passing logic. Enables runtime polymorphism, one of the core pillars that differentiates object-oriented systems from purely procedural ones.

Q9. Construct a Concept Map — Software Development Models

5 Marks

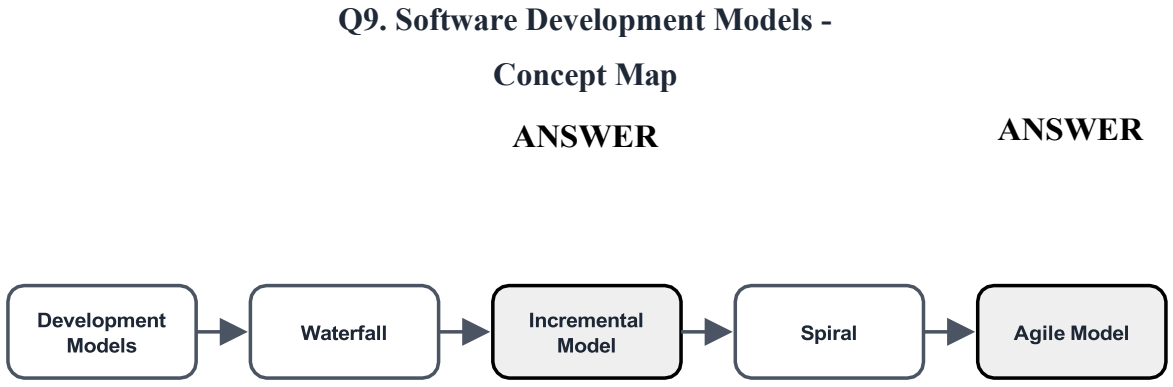
GIVEN

```
Software Development Models → Waterfall → _______ → Spiral → _______
```

ANSWER

Missing concept(s): Incremental Model, Agile Model.

CONCEPT MAP DIAGRAM



□ *Given concept* □ *Answer (missing concept filled in)*

EXPLANATION

The **Incremental Model** divides the system into small functional builds/increments, each of which is designed, coded, and tested and then delivered one after another, gradually adding features until the full system is complete. The **Agile Model** is an iterative approach built around short cycles (sprints), continuous customer collaboration, and the ability to adapt requirements even late in development.

The Waterfall Model is the simplest, purely sequential model — each phase must fully finish before the next begins, with no overlap or going back. The Incremental Model relaxes this rigidity by delivering usable pieces of software early and often, reducing the risk of building the wrong thing over a long single cycle. The Spiral Model combines iterative development with a strong emphasis

on formal risk analysis at every loop ('spiral') of the project. The Agile Model takes iteration even further, prioritizing working software, customer feedback, and responsiveness to change over rigid up-front planning.

WORKED EXAMPLE

A government e-passport project with fixed, unchanging regulations might use the Waterfall Model, completing each phase fully before moving on. A retail company building a shopping app might use the Incremental Model, releasing the product-browsing feature first, then adding cart and payment in later builds. A large aerospace control system, where risk of failure is very costly, might use the Spiral Model to repeatedly assess risk at each stage. A startup building a social media feature might use the Agile Model, running two-week sprints and adjusting the feature based on weekly user feedback.

ANALYSIS

Enhances adaptability to changing requirements compared to rigid sequential models. Reduces project risk by delivering and validating small pieces of the system early rather than waiting until the very end. Improves customer satisfaction through frequent delivery of working software and continuous feedback loops.

Q10. Construct a Concept Map — Class Diagram Elements

5 Marks

GIVEN

```
Class Diagram → Class → _____ → Association → _____
```

ANSWER

Missing concept(s): Attribute, Method.

CONCEPT MAP DIAGRAM

Q10. Class Diagram Elements -

Concept Map

ANSWER **ANSWER**

□ *Given concept* □ *Answer (missing concept filled in)*

EXPLANATION

An **Attribute** is a property/field defined inside a Class in a Class Diagram, representing a characteristic or piece of data that every object of that class will hold (shown in the middle compartment of the class box). A **Method** is an operation defined inside a Class in a Class Diagram, representing an action or behavior the class can perform (shown in the bottom compartment of the class box).

A Class Diagram visually defines a Class using three compartments: the class name, its Attributes, and its Methods. Association is a structural relationship shown as a line connecting two classes, indicating that objects of one class are connected to (know about, or use) objects of another class.

Attributes and Methods define what a single class looks like internally, while Association defines how multiple classes relate to one another externally.

WORKED EXAMPLE

In a Class Diagram for an e-commerce system, the Order class box would list Attributes such as orderId and orderDate in its middle compartment, and Methods such as calculateTotal() and cancelOrder() in its bottom compartment. An Association line connecting Order to Customer, labelled 'places', shows that a Customer object places one or more Order objects — this relationship is what the exam's blank labelled Association is testing understanding of.

ANALYSIS

Represents object structure effectively, at both the individual class level (attributes/methods) and the system level (associations). Improves software documentation, since developers can read the class diagram to understand data and behavior without reading all the code. Supports object-oriented design decisions by making relationships and responsibilities explicit before coding begins.

Q11. Construct a Concept Map — Software Engineering Principles *5 Marks*

GIVEN

```
Software Engineering Principles → Modularity → _____ → Reusability → _____
```

ANSWER

Missing concept(s): Maintainability, Scalability.

CONCEPT MAP DIAGRAM

Q11. Software Engineering Principles - Concept Map

ANSWER **ANSWER**

□ *Given concept* □ *Answer (missing concept filled in)*

EXPLANATION

Maintainability follows naturally from Modularity — because the system is broken into independent, well-defined modules, developers can locate and fix issues, or add features, in one module without having to fully understand or disturb the rest of the system. **Scalability** follows from Reusability — because well-designed, reusable components can be combined and multiplied, the system can be extended to handle more users, data, or load without a complete redesign.

Modularity breaks a large system into smaller, independent, manageable units (modules/classes/components) — this is the foundation principle. Once modules are well-defined, they naturally become easier to maintain (Maintainability), since changes stay localized. Well-

built modules can also be reused across different parts of the system or even in different projects (Reusability), saving development effort. Reusable, well-modularized components allow the overall system to grow and handle increased demand more gracefully (Scalability).

WORKED EXAMPLE

In a school management system, Modularity is applied by separating the system into independent modules: Attendance, Grading, and Fee Management. Because each module is independent, Maintainability follows — if the Fee Management module needs a new late-fee rule, developers only touch that module. The Grading module's report-generation component may then be reused (Reusability) inside a separate Exam Portal project without rewriting it. Finally, because the modules are loosely coupled and reusable, the school district can later add ten more schools to the same system (Scalability) without redesigning the whole architecture.

ANALYSIS

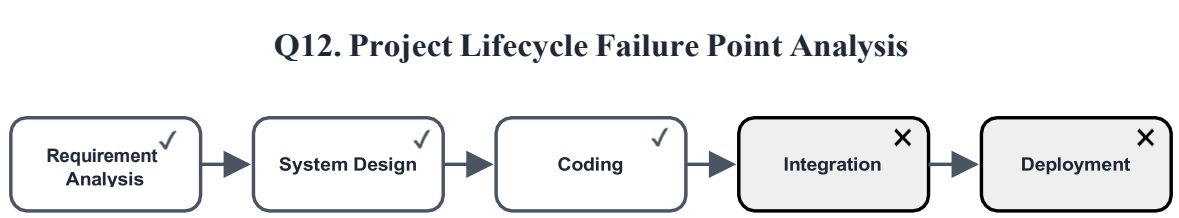
Improves overall software performance and structure by avoiding tightly-coupled, monolithic code. Reduces maintenance effort and cost over the software's lifetime. Supports long-term software evolution, allowing the system to grow with business needs instead of being rewritten from scratch.

Q12. Concept Map with Failure Analysis — Project Lifecycle Breakdown *5 Marks*

GIVEN

```
Requirement Analysis ✓ → System Design ✓ → Coding ✓ → Integration ✗ → Deployment ✗
```

CONCEPT MAP DIAGRAM



□ *Phase completed (check)* □ *Phase failed / blocked (cross)*

EXPLANATION

The check marks confirm that Requirement Analysis, System Design, and Coding were all completed successfully — individually, each module was correctly specified, designed, and implemented. The first cross mark appears at **Integration**: this is where individually-working modules are combined into a single working system, and here the failure occurs — the modules do not communicate or interoperate correctly with each other (e.g., mismatched interfaces, incompatible data formats, or broken API contracts). Since Integration failed, Deployment is blocked as well, because a system whose parts do not work together cannot be safely released to production.

WORKED EXAMPLE

Consider an online exam platform built as three separately-coded modules: a Login module, a

Question-Bank module, and a Result module. Each module individually passes its own unit tests. However, when combined, the Login module sends a userId as an integer while the Result module expects it as a string, so results fail to save against the correct student — a classic Integration-phase failure even though every module was 'correct' on its own.

ANALYSIS

Integration failure causes functional inconsistencies — features that worked in isolation break when combined. It delays the overall software deployment timeline, since the team must go back and reconcile interfaces between modules. It increases debugging effort significantly, because the root cause could be in any one of several previously 'working' modules.

SOLUTION / CORRECTIVE ACTION

Perform dedicated integration testing (not just unit testing) once individual modules are complete. Verify module interfaces/APIs and data contracts explicitly before combining modules, ideally against a written interface specification. Resolve compatibility issues (data formats, protocols, versioning) methodically before attempting final Deployment.